

Formal Methods as the Harness for AIOS

Mingyu Li Zhanbo Wang Yiping Weng

Key Laboratory of System Software (Chinese Academy of Sciences)
Institute of Software, Chinese Academy of Sciences
University of Chinese Academy of Sciences

Abstract

AI-generated OSES outpace human review. Trust must now be sustained in continuously evolving, non-human logic. We present SPECOS, an OS design grounded in relational state, functional logic, and two-phase verification, to enable formal assurance to match the speed of AI code generation.

1 A Trust Crisis in Generative AIOS

Each leap in OS designs has been a direct response to the bottleneck of its era: monolithic kernels [22] introduced POSIX [25] to abstract hardware heterogeneity; microkernels [15] minimized the TCB to isolate software faults; exokernels [10] exposed raw resources to bypass inefficient abstractions; and multikernels [5] treated the machine as a network to overcome multicore scaling limits.

However, these OS designs share an obsolescent premise: that the system logic is a human artifact, curated at human timescales, and updated through human review. The AI era shatters this assumption. LLMs can now synthesize new OS functions, policies, and translate human intents into tasks within minutes [9, 16, 28]. This velocity creates a fundamental verification gap.

Safety alignment via prompts cannot close this gap: hard invariants, such as bounded resource consumption and no privilege escalation, require formal proof, not probabilistic assertions. Yet, existing formally verified kernels, relying either on manual theorem proving (seL4 [14], CertiKOS [12]) or automatic SMT solving (Hyperkernel [20]), are tailored for static, human-curated codebases. These systems lack the architectural agility to continuously ingest and verify high-velocity, AI-generated OS logic at machine timescales. Meanwhile, SysSpec [16] pioneers a specification-guided evolution approach for AI-driven generative filesystems, but its correctness still hinges on empirical testing. Without machine-checkable formal semantics and rigorous proof obligations, SysSpec falls short in delivering strong safety assurance.

SPECOS is designed for this new reality: the OS does not just execute AI-generated code, but autonomously and incrementally verifies its correctness as it evolves in real-time.

2 SPECOS: Verification by Construction

SPECOS aims to satisfy two key requirements:

- **Soundness:** every AI-generated component must be formally verified before execution.
- **Incrementality:** only localized subgraph re-verification is needed for machine-speed evolution.

To accomplish this, SPECOS introduces three architectural primitives that make verifiability an intrinsic property of the system rather than an afterthought.

2.1 State as Relational Tables

SPECOS represents all system state, including hardware resources, task metadata and capabilities, as relational tables. This design choice provides three advantages.

Automated Invariant Enforcement. Because first-order logic over concrete finite relations is decidable [3], OS invariants expressed as relational constraints can be enforced directly by the SPECOS’s state management engine.

Controlled Symbolic Space. Relational tables constrain system state to a bounded, structured fact space, laying the foundation for the polynomial-time guarantee of the third primitive (see §2.3, two-phase verification).

Transactional Isolation. By adopting ACID properties [11], we map classic database discipline to AIOS safety. Related ideas have appeared in DBOS [23]; SPECOS further observes that this choice is well aligned with agentic execution: *atomicity* ensures that defective AI logic cannot leave the system in a partial or contaminated state; and *isolation* allows concurrent AI agents to operate without interference [6].

2.2 Logic as Functional Lambdas

AI-generated logic (e.g., OS services and tasks) is implemented as pure, memory-safe functional lambdas (e.g., OCaml [2] as in MirageOS [18], Haskell [1] as in House [13]):

$$\lambda : \text{State} \xrightarrow{\Sigma} \text{Event} \rightarrow (\text{State}', \text{Intent})$$

Here, Σ defines the *effect signature* (permitted table access), and Intent is a closed algebraic type representing logical requests rather than raw physical commands. This model enforces total concern separation: components maintain no internal mutable state, and all persistent effects are mediated through relational tables.

Compositional Safety. Under SPECOS’s effect discipline, two functional lambdas f and g compose ($g \circ f$) only if their write sets (Σ_f, Σ_g) are non-conflicting and their event types are compatible [27]. This allows SPECOS to preserve topological invariants via incremental fixed-point propagation [26].

Deferred Side Effects. To prevent irreversible consequences (e.g., sending network packets or issuing sensor commands), SPECOS buffers Intent sequences produced by executing a

lambda DAG in isolated transactions. Only after the entire DAG passes global verification and the transaction commits successfully does SPECOS materialize those intents into the physical world.

2.3 Two-Phase Verification

Semantic verification of AI-generated lambda code is undecidable in general cases [21]. SPECOS decomposes the verification problem into two tractable subproblems.

Local Verification. Before deployment, each AI-generated lambda undergoes bounded model checking (BMC) to extract a behavioral summary (including Σ , output types, and resource bounds). Although BMC is traditionally unsound due to under-approximation, SPECOS achieves *soundness* by ensuring lambdas are loop-bounded by construction (akin to eBPF [24]) and restricted to finite relational domains.

Global Verification. SPECOS evaluates whether summary compositions preserve system-level invariants, which are expert-curated specifications as the root of trust. Expressed in a decidable DSL (e.g., Datalog [19]), these invariants enforce properties like liveness and real-time safety. Computing the invariants reduces to fixed-point evaluation over finite relations, ensuring polynomial-time verification.

3 Case Study: ROS-Next

Embodied AI represents the most critical scenario for AIOS, where the trust vacuum carries physical risks. By replacing the industry standard ROS2 control plane [17] with SPECOS, we show that continuous formal verification can be embedded into robotic stacks without sacrificing real-time performance. Table 1 summarizes the main differences.

ROS2 Limitations. ROS2’s decentralized pub-sub architecture fragments control state across nodes. Lacking global atomicity, asynchronous message delays cause navigation and planning nodes to act on inconsistent pose snapshots, yielding stale execution intents [4]. More critically, AI-evolved behavior trees [8] are currently validated through empirical simulation, an inherently incomplete approach that cannot keep pace with real-time logic synthesis.

Control-plane consistency. SPECOS represents the robot’s entire control state (e.g., pose, velocity, joint state) as centralized relational tables. All lambda components read globally consistent snapshots of control state through MVCC [6], eliminating state tearing at the architectural level. When nodes fail, control-plane transactions support atomic roll-back, ensuring the robot never persists an invalid state.

Verifiable Task Decisions. High-level tasks in ROS2, such as behavior trees or task graphs, used to depend heavily on simulation and manual debugging. SPECOS defines all actions as lambdas, compiles human intent into lambda DAGs, and performs two-phase verification before deployment. First, BMC extracts each lambda’s effect signature Σ , event type,

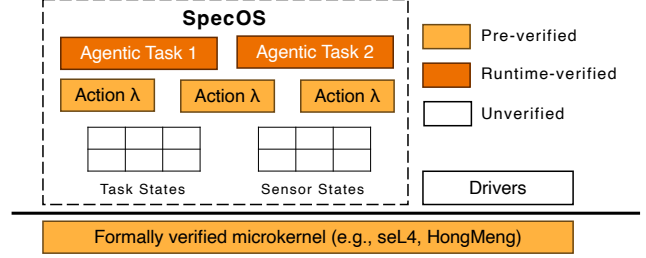


Figure 1. SPECOS overview for Embodied AI. Both the microkernel and robotic action lambdas can be pre-verified offline; only AI-synthesized tasks (DAGs of lambdas) require online verification.

and resource bound. Then, SPECOS verifies the entire DAG invariants (e.g., acyclicity and capability confinement) in polynomial time. Topology updates of a behavior tree only trigger re-verification of the affected subgraph.

Layered Verification. To guarantee strict *real-time invariants*, SPECOS bifurcates verification by frequency:

- **High-Frequency Layer:** Low-level control loops (≥ 100 Hz) use static topologies verified *offline* (pre-deployment).
- **Low-Frequency Layer:** High-level planning and AI-generated agentive tasks (3–10 Hz) undergo the full *runtime* two-phase verification before their intents are materialized.

Both layers preserve soundness; the difference lies only in the time at which verification occurs.

Relationship to Verified Kernels. As illustrated in Figure 1, SPECOS builds on a formally verified microkernel (e.g., seL4 [14] or HongMeng [7]) to isolate unverified drivers. Traditional theorem provers secure the stable, human-authored core, while our two-phase verification targets only the high-velocity, AI-generated components. This decoupling concentrates incremental verification where the system evolves rapidly.

4 Conclusion

Historically, OS design focused on resource sandboxing via hardware (MMU, MPU) or SFI. However, AIOS demands *semantic sandboxing*. SPECOS achieves this by treating incremental formal verification as a first-class OS property.

Acknowledgments

This work was partially funded by the CAS Talent Introduction Project, Institute of Software, Chinese Academy of Sciences (No. E5YD230101), and the National Natural Science Foundation of China (No. 62502510).

References

- [1] 2026. Haskell. <https://www.haskell.org/>
- [2] 2026. OCaml: An industrial-strength functional programming language with an emphasis on expressiveness and safety. <https://ocaml.org/>
- [3] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Addison-Wesley.

Table 1. Comparison between ROS2 and SPECOS.

	ROS2	SPECOS
Consistency model	Distributed local control state	Centralized state in relational tables
Fault isolation	Crash recovery by node restart	Atomic rollback of control-plane transactions
Side-effect isolation	Side effects execute directly	Intent buffering + layered two-phase commit
High-frequency control	Offline test and manual validation	Pre-deployment formal verification
AI-evolved behavior trees	No runtime guarantees	Incremental re-verification on topology change

- [4] Soroush Bateni, Marten Lohstroh, Hou Seng Wong, Hokeun Kim, Shaokai Lin, Christian Menard, and Edward A. Lee. 2023. Risk and Mitigation of Nondeterminism in Distributed Cyber-Physical Systems. In *21st ACM-IEEE International Symposium on Formal Methods and Models for System Design, MEMOCODE 2023, Hamburg, Germany, September 21-22, 2023*. ACM / IEEE, 1–11.
- [5] Andrew Baumann, Paul Barham, Pierre-Evariste Dagand, Tim Harris, Rebecca Isaacs, Simon Peter, Timothy Roscoe, Adrian Schüpbach, and Akhilesh Singhanian. 2009. The multikernel: a new OS architecture for scalable multicore systems. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles* (Big Sky, Montana, USA) (*SOSP '09*). Association for Computing Machinery, New York, NY, USA, 29–44.
- [6] Philip A. Bernstein and Nathan Goodman. 1983. Multiversion concurrency control—theory and algorithms. *ACM Trans. Database Syst.* 8, 4 (Dec. 1983), 465–483.
- [7] Haibo Chen, Xie Miao, Ning Jia, Nan Wang, Yu Li, Nian Liu, Yutao Liu, Fei Wang, Qiang Huang, Kun Li, Hongyang Yang, Hui Wang, Jie Yin, Yu Peng, and Fengwei Xu. 2024. Microkernel Goes General: Performance and Compatibility in the HongMeng Production Microkernel. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 465–485.
- [8] Michele Colledanchise and Petter Ögren. 2018. Behavior Trees in Robotics and AI.
- [9] Jonathan Corbet. 2025. Supporting kernel development with large language models. <https://lwn.net/Articles/1026558/>
- [10] D. R. Engler, M. F. Kaashoek, and J. O'Toole. 1995. Exokernel: an operating system architecture for application-level resource management. In *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles* (Copper Mountain, Colorado, USA) (*SOSP '95*). Association for Computing Machinery, New York, NY, USA, 251–266.
- [11] Jim Gray. 1981. The transaction concept: virtues and limitations (invited paper). In *Proceedings of the Seventh International Conference on Very Large Data Bases - Volume 7* (Cannes, France) (*VLDB '81*). VLDB Endowment, 144–154.
- [12] Ronghui Gu, Zhong Shao, Hao Chen, Xiongnan (Newman) Wu, Jieung Kim, Vilhelm Sjöberg, and David Costanzo. 2016. CertiKOS: An Extensible Architecture for Building Certified Concurrent OS Kernels. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. USENIX Association, Savannah, GA, 653–669.
- [13] Thomas Hallgren, Mark P. Jones, Rebekah Leslie, and Andrew Tolmach. 2005. A principled approach to operating system construction in Haskell. *SIGPLAN Not.* 40, 9 (Sept. 2005), 116–128.
- [14] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. 2009. seL4: formal verification of an OS kernel. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles* (Big Sky, Montana, USA) (*SOSP '09*). Association for Computing Machinery, New York, NY, USA, 207–220.
- [15] J. Liedtke. 1995. On micro-kernel construction. *SIGOPS Oper. Syst. Rev.* 29, 5 (Dec. 1995), 237–250.
- [16] Qingyuan Liu, Mo Zou, Hengbin Zhang, Dong Du, Yubin Xia, and Haibo Chen. 2026. Sharpen the Spec, Cut the Code: A Case for Generative File System with SYSSPEC. In *24th USENIX Conference on File and Storage Technologies (FAST 26)*. USENIX Association, Santa Clara, CA, 291–311.
- [17] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. 2022. Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics* 7, 66 (2022), eabm6074. [arXiv:https://www.science.org/doi/pdf/10.1126/scirobotics.abm6074](https://www.science.org/doi/pdf/10.1126/scirobotics.abm6074)
- [18] Anil Madhavapeddy, Richard Mortier, Charalampos Rotsos, David Scott, Balraj Singh, Thomas Gazagnaire, Steven Smith, Steven Hand, and Jon Crowcroft. 2013. Unikernels: library operating systems for the cloud. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems* (Houston, Texas, USA) (*ASPLOS '13*). Association for Computing Machinery, New York, NY, USA, 461–472.
- [19] David Maier, K. Tuncay Tekle, Michael Kifer, and David S. Warren. 2018. *Datalog: concepts, history, and outlook*. Association for Computing Machinery and Morgan & Claypool, 3–100.
- [20] Luke Nelson, Helgi Sigurbjarnarson, Kaiyuan Zhang, Dylan Johnson, James Bornholt, Emina Torlak, and Xi Wang. 2017. Hyperkernel: Push-Button Verification of an OS Kernel. In *Proceedings of the 26th Symposium on Operating Systems Principles* (Shanghai, China) (*SOSP '17*). Association for Computing Machinery, New York, NY, USA, 252–269.
- [21] H. G. Rice. 1953. Classes of Recursively Enumerable Sets and Their Decision Problems. *Trans. Amer. Math. Soc.* 74, 2 (1953), 358–366. <http://www.jstor.org/stable/1990888>
- [22] Dennis M. Ritchie and Ken Thompson. 1974. The UNIX time-sharing system. *Commun. ACM* 17, 7 (July 1974), 365–375.
- [23] Athinagoras Skiadopoulos, Qian Li, Peter Kraft, Kostis Kaffes, Daniel Hong, Shana Mathew, David Bestor, Michael Cafarella, Vijay Gade-pally, Goetz Graefe, Jeremy Kepner, Christos Kozyrakis, Tim Kraska, Michael Stonebraker, Lalith Suresh, and Matei Zaharia. 2021. DBOS: a DBMS-oriented operating system. *Proc. VLDB Endow.* 15, 1 (Sept. 2021), 21–30.
- [24] Dave Thaler. 2024. BPF Instruction Set Architecture (ISA). <https://www.rfc-editor.org/rfc/rfc9669.html>
- [25] The Open Group. 2018. The Open Group Base Specifications Issue 7, IEEE Std 1003.1-2008. <https://pubs.opengroup.org/onlinepubs/9699919799/> Accessed 2026-07-01.
- [26] J. D. Ullman. 1989. Bottom-up beats top-down for datalog. In *Proceedings of the Eighth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems* (Philadelphia, Pennsylvania, USA) (*PODS '89*). Association for Computing Machinery, New York, NY, USA, 140–149.
- [27] George Zakhour, Pascal Weisenburger, and Guido Salvaneschi. 2024. Automated Verification of Fundamental Algebraic Laws. *Proc. ACM Program. Lang.* 8, PLDI, Article 178 (June 2024), 24 pages.
- [28] Yusheng Zheng, Yiwei Yang, Maolin Chen, and Andrew Quinn. 2024. Kgent: Kernel Extensions Large Language Model Agent. In *Proceedings of the ACM SIGCOMM 2024 Workshop on EBPF and Kernel Extensions* (Sydney, NSW, Australia) (*eBPF '24*). Association for Computing Machinery, New York, NY, USA, 30–36.